

Fullstack Development

API Architectures and Design #2

Content

- What is API?
- API Architecture Styles
- **RESTful API design**
- API Security
- API Testing

RESTful API Design

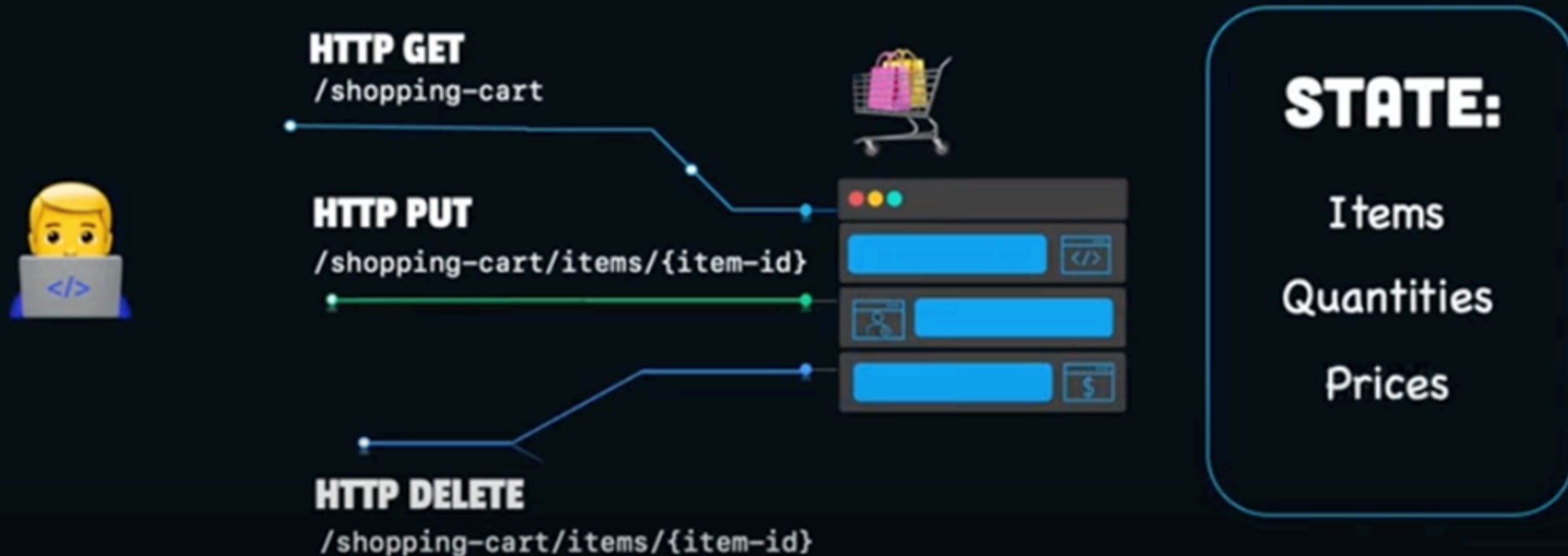
RESTful API Design

Stateless

- **Single-server** environment
 - Server process any request without knowledge of previous requests
- **Multi-server** environment
 - Request not bound to a specific server
 - Able to provide service in `load-balanced` fashion
- Make APIs more `scalable` > more available
- Easy to `cache` (without state info)

RESTful API Design (2)

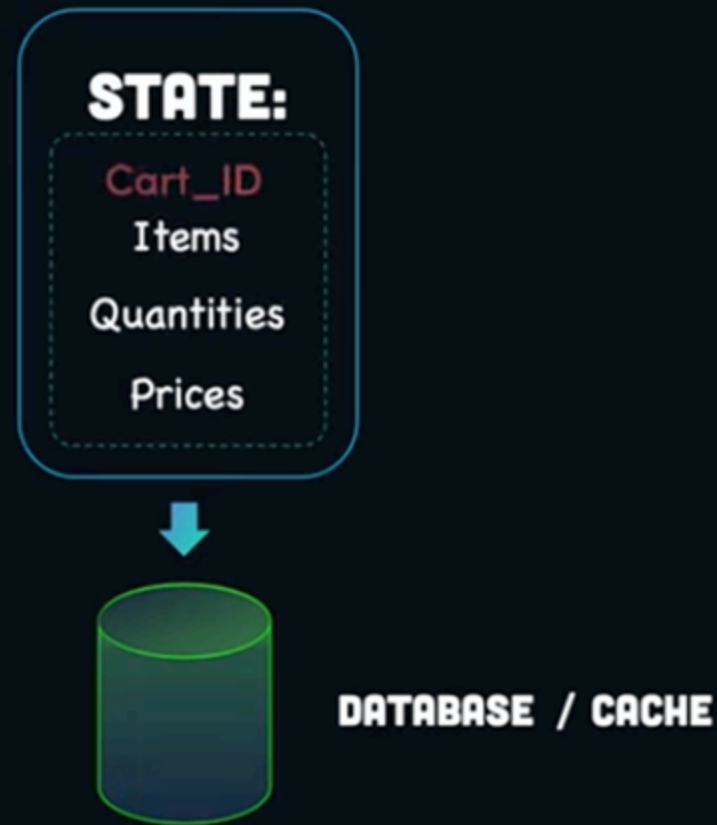
Most applications require some forms of state information



RESTful API Design (3)

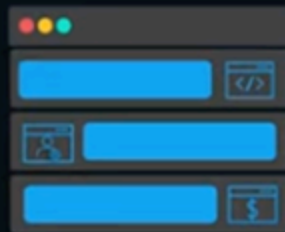
2. STORE THE STATE EXTERNALLY

2.1. USE UNIQUE IDENTIFIERS



RESTful API Design (4)

3. USE IDS TO GET THE
STATE OF THE CART



Cookie: Cart_ID

STATE:

Cart_ID

Items

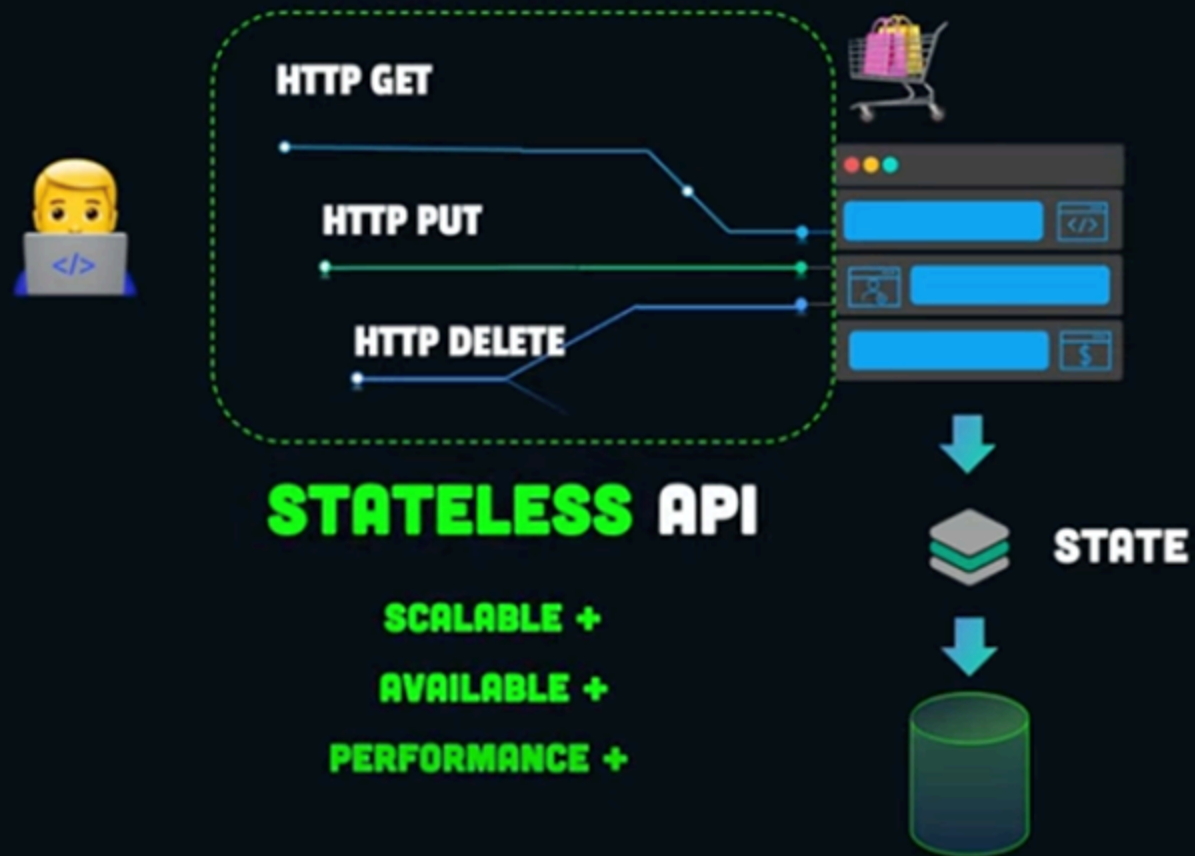
Quantities

Prices



DATABASE / CACHE

RESTful API Design (5)



RESTful API Design (6)

Organize API around **resources** (not actions/verbs)

`https://e-commerce.com/orders` // Good → **RESOURCE**

`https://e-commerce.com/generate-order` // Avoid → **ACTION**

`https://e-commerce.com/users` // Good → **RESOURCE**

`https://e-commerce.com/create-user` // Avoid → **ACTION**

Using **HTTP method** to differentiate **actions**

RESTful API Design (7)

Endpoint consistency is key, using plural form in URLs is recommended

Create order



+

<https://e-commerce.com/orders>

<https://e-commerce.com/initiateOrder>

<https://e-commerce.com/makeOrder>

<https://e-commerce.com/conceiveOrder>

<https://e-commerce.com/fatherOrder>

RESTful API Design (8)

HTTP Methods and CRUD operations



RESTful API Design (9)

How to do complex query?

Q: GET ALL ORDERS MADE BY A CUSTOMER 

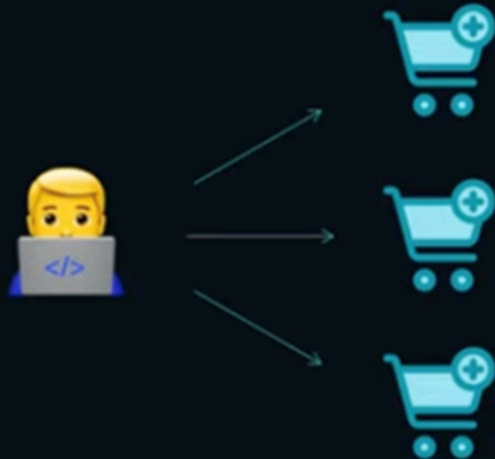
+ SORTED

+ PAGINATED

RESTful API Design (10)

Entities are grouped into collections

HTTP GET  /customers/customer/orders



Tip 1 Entities are grouped into collections.

-> intuitive API

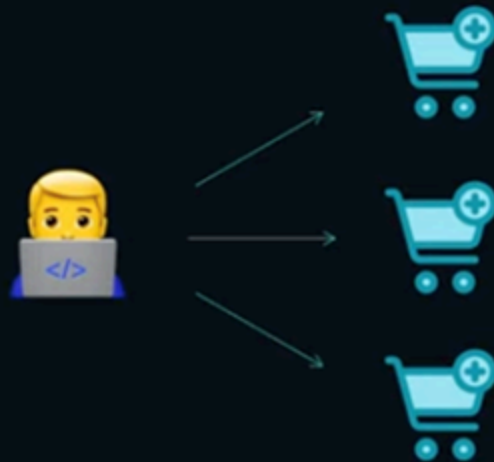
-> consistent naming convention

RESTful API Design (11)

Use **parameterized URIs** for identity

`/customers/orders?customer_id=1` 🙄

HTTP GET `/customers/{customer_id}/orders`



Tip 2 Use parametrized URIs for identity.

RESTful API Design (12)

Avoid deeply nested URLs (`/collection/item/collection`)

HTTP GET `/customers/{customer_id}/orders`



Tip 3 Avoid resource URLs more complex than `/collection/item/collection`

`/customers/1/orders/99/products` 🙅

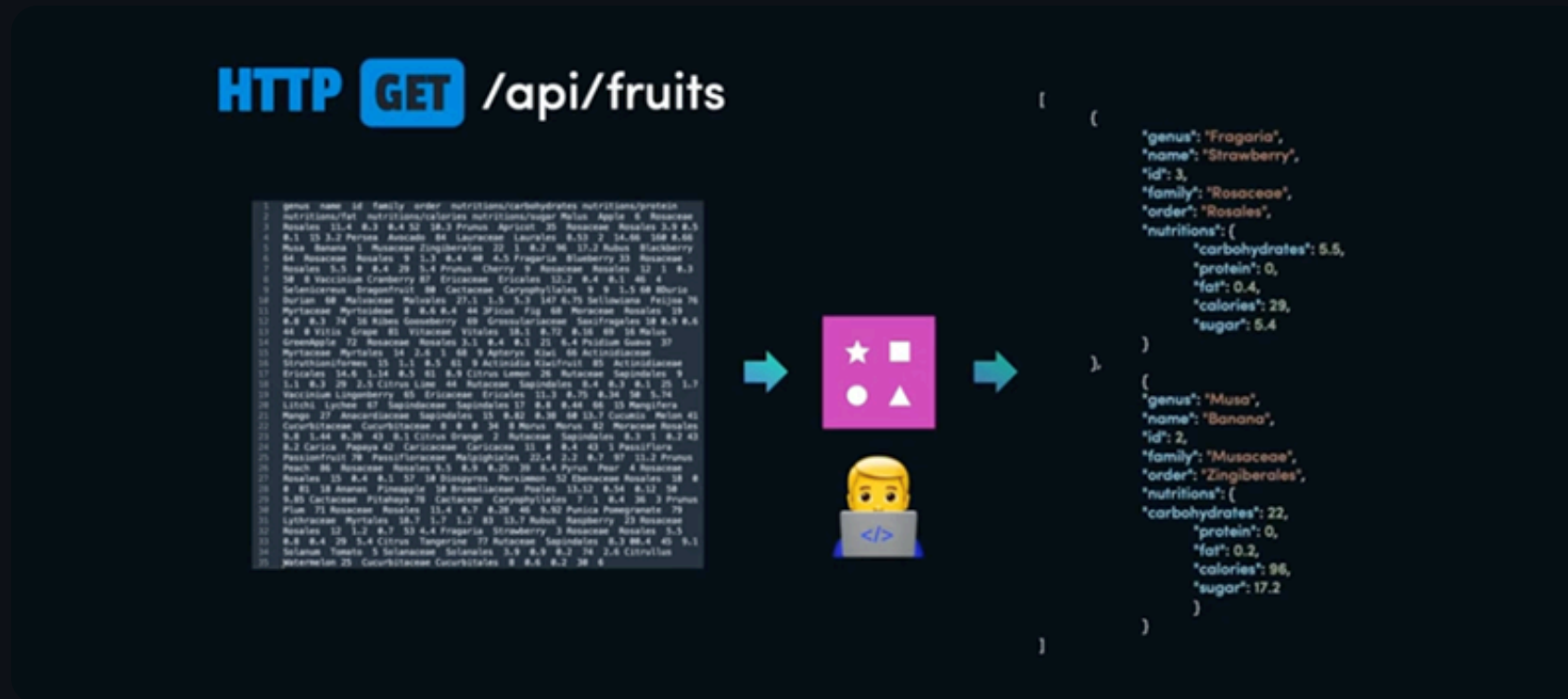
`/customers/1/orders` 👍
`/orders/99/products`

GET `/customers/1?order=99`

RESTful API Design (14)

Do not return plain-text response

- Client has to do some extra parsing/processing



RESTful API Design (15)

Response with **JSON** / **XML** / **YAML**



JSON

```
{
  "configurations": [
    {
      "name": "Ingress",
      "value": "data/input"
    },
    {
      "name": "Egress",
      "value": "data/output"
    }
  ]
}
```



XML

```
<Configurations>
  <Config>
    <Name>Ingress</Name>
    <Value>data/input</Value>
  </Config>
  <Config>
    <Name>Egress</Name>
    <Value>data/output</Value>
  </Config>
</Configurations>
```



YAML

```
---
configurations:
- name: Ingress
  value: data/input
- name: Egress
  value: data/output
```

RESTful API Design (16)

Allow client to specify **Content-Type** header

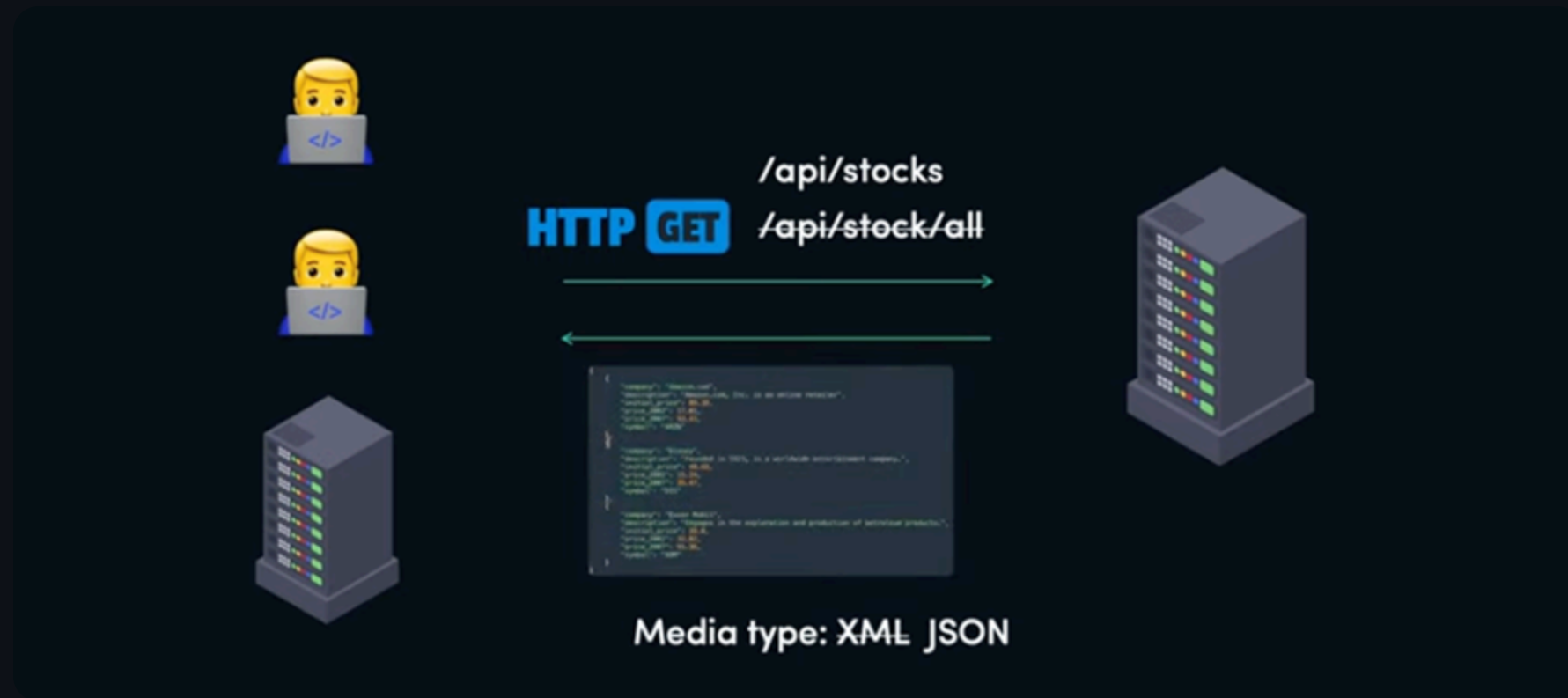
HTTP **Header**

Content-Type : application/json



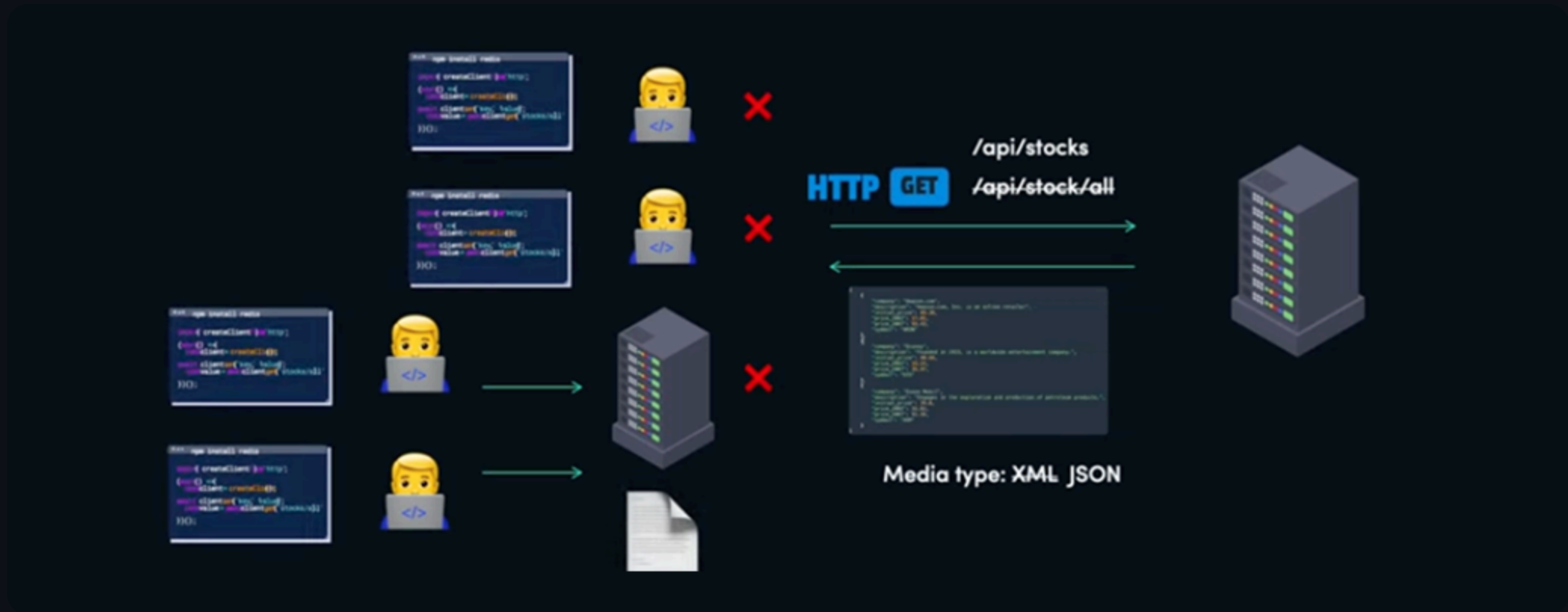
RESTful API Design (17)

Changing API after it has been adopted is dangerous!



RESTful API Design (18)

Clients need to **update code/documents** to make app work again



RESTful API Design (19)

However, business requirements are always changed

HTTP GET /api/stocks

Add new field
→

Change field to list
→

Change media type
→

etc

```
{
  "company": "Amazon.com",
  "description": "Amazon.com, Inc. is an online retailer",
  "initial_price": 89.38,
  "price_2002": 17.01,
  "price_2007": 93.43,
  "symbol": "AMZN"
},
{
  "company": "Disney",
  "description": "Founded in 1923, is a worldwide entertainment company.",
  "initial_price": 40.60,
  "price_2002": 15.24,
  "price_2007": 35.47,
  "symbol": "DIS"
},
{
  "company": "Exxon Mobil",
  "description": "Engages in the exploration and production of petroleum products.",
  "initial_price": 39.0,
  "price_2002": 32.02,
  "price_2007": 91.36,
  "symbol": "XOM"
}
}
```

RESTful API Design (20)

API versioning is the answer

API VERSIONING

URI

`https://e-commerce.com/v2/customers/`

QUERY PARAMS

`https://e-commerce.com/customers?version=2`

HEADERS

`https://e-commerce.com/customers`

`Custom-Header: api-version=2`

MEDIA TYPE

`https://e-commerce.com/customers`

`Accept: application/e-commerce.v1+json`

RESTful API Design (21)

API versioning comparison

	CACHE-FRIENDLY	RESTFUL
URI	✓	
QUERY PARAMS	✓	
HEADERS		✓
MEDIA TYPE		✓

RESTful API Design (22)

API exception handling and response with proper HTTP status code

WTF?

HTTP GET /api/users/{id}



HTTP 500 Internal Server Error

An error occurred!

Get user info

```
app.get('/api/users/:id', function(req, res) {  
  // Some code that throws an Exception  
  throw new Error('An error occurred!');  
});
```



HTTP GET /api/users/{id}

HTTP 400 Bad Request

'Invalid data type for
the "id" parameter

Get user info

```
app.get('/api/users/:id', function(req, res) {  
  try {  
    throw new Error('Invalid data type for the "id" parameter');  
  } catch (error) {  
    res.status(400).json({ error: 'Bad Request', message: error.message });  
  }  
});
```

RESTful API Design (23)

HTTP status code



4xx

- 400** Bad Request
- 401** Unauthorized
- 403** Forbidden
- 404** Not Found
- 405** Method Not Allowed
- 406** Not Acceptable
- 404** Not Found
- 409** Conflict
- 423** Locked



5xx

- 500** Internal Server Error
- 503** Service Unavailable



RESTful API Design (24)

Including links in JSON responses is a common practice

```
{
  "id": 1,
  "name": "Example",
  "links": {
    "self": "http://api.example.com/resource/1",
    "related": "http://api.example.com/resource/2"
  }
}
```

RESTful API Design (25)

Content return policies

Return the created resource

- HTTP status code: 201 Created, Location header of resource's URI
- Full detail of the newly created resource

```
HTTP/1.1 201 Created
Location: /resources/123
Content-Type: application/json
{
  "id": 123,
  "name": "New Resource"
}
```

RESTful API Design (26)

Content return policies

Do not return content

- Only a `201 Created` or `204 No Content` response, and `Location` header
- Minimizes data transfer

```
HTTP/1.1 201 Created  
Location: /resources/123
```

References

- [Best practices for REST API design](#)
- [Mastering RESTful API Design: A Practical Guide](#)
- [REST API - Best Practices - Design](#)